

UNIT- 5

INTERMEDIATE SQL

What is the index?

The index is responsible for the performance optimization of the database, so the proper use of the index can help the database improve performance substantially and the related user experience.

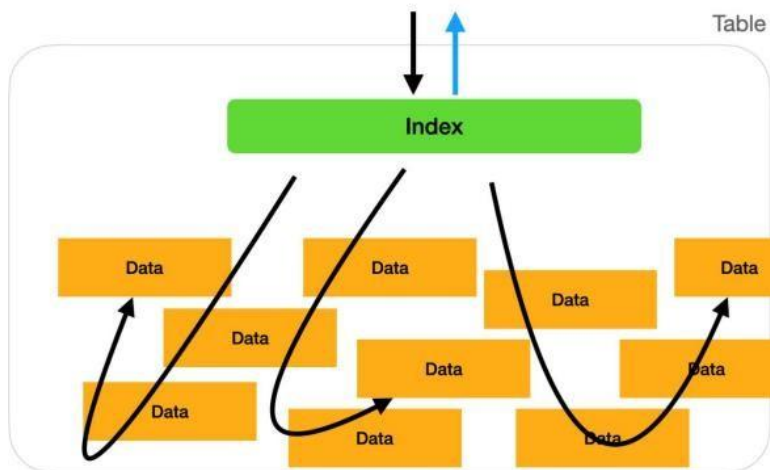
Understand Index:

Concept & Pros:

The core concept of index technology is to mark specific columns in the table and extract them to be stored additionally and in an organized manner.

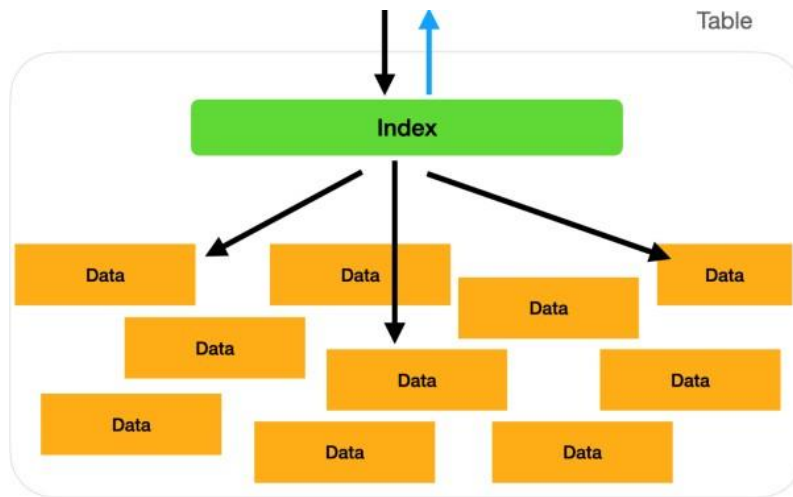
Without INDEX:

In the index layer, no index is found, so many redundant query paths may be taken to retrieve data.



With index:

In the index layer, if an index is found and it has relative information to your query, then you can directly know which data to fetch without taking a detour.



Syntax:

The syntax for adding an index is very simple, similar to the syntax for adding FOREIGN KEY

```
ALTER TABLE `new_schema`.`users`  
  ADD INDEX `name_index` (`name`);
```

Categories:

1. Primary Index (Primary Key)

- Uniquely identifies each row in a table.
- Cannot contain duplicate values or NULL values.
- A table can have only one primary key.

Example:

```
CREATE TABLE Student ( id  
INT PRIMARY KEY,  name  
VARCHAR(50)  
);
```

id is the primary key and uniquely identifies each student.

2. Unique Index

- Ensures that all values in a column (or combination of columns) are unique.
- Unlike a primary key, a unique index can allow NULL values, depending on the column definition and database rules.

Example:

```
CREATE UNIQUE INDEX idx_email  
ON Student(email);
```

This prevents duplicate non-NULL email addresses.

3.Full-Text Index

- Used for searching words and phrases in text columns.
- Commonly used for articles, descriptions, and other large text fields.

Example:

```
CREATE FULLTEXT INDEX idx_description  
ON Articles(description);
```

Six Best Practices for Indexing in MySQL

1. **Use UNIQUE INDEX:** Prefer a unique index when values must be unique; otherwise, use a normal index.
2. **Index frequently used columns:** Add indexes to columns often used in GROUP BY, ORDER BY, and joins.
3. **Index frequently searched columns:** Add indexes to columns commonly used in WHERE with many distinct values, such as an ID number.
4. **Save disk space:** Indexes use storage space, so create only necessary indexes.
5. **Avoid indexing large text columns unnecessarily:** Index short columns, such as titles, when appropriate, instead of large content fields.
6. **Be careful with NULL values:** Do not avoid indexing a column just because it contains NULL; index it when query patterns and performance justify it.